

15 – Recursion

RECURSIÓN Y VARIABLES GLOBALES

Es posible que hayas escuchado sobre **recursión**, y muy probablemente pienses en ella como una técnica de programación bastante sutil. Esa es una leyenda urbana encantadora difundida por científicos de la computación para hacer pensar a la gente que somos más inteligentes de lo que realmente somos. La recursión es una idea importante, pero no es tan sutil, y es **más que** una técnica de programación.

Como método descriptivo, la recursión es ampliamente utilizada, incluso por personas que nunca soñarían con escribir un programa. Considera parte del código legal de los Estados Unidos que define la noción de ciudadanía por "derecho de nacimiento". En términos generales, la definición es la siguiente:

- Cualquier niño nacido dentro de los Estados Unidos o
- Cualquier niño nacido en situación irregular fuera de los Estados Unidos, uno de cuyos padres sea ciudadano de los Estados Unidos.

La primera parte es simple; si naces en los Estados Unidos, eres ciudadano por derecho de nacimiento (como Barack Obama). Si no naces en EE.UU., depende de si tus padres eran ciudadanos estadounidenses en el momento de tu nacimiento. Y si tus padres eran ciudadanos estadounidenses podría depender de si los padres de ellos eran ciudadanos estadounidenses, y así sucesivamente.

En general, una definición recursiva está compuesta de dos partes. Hay al menos un **caso base** que especifica directamente el resultado para un caso especial (caso 1 en el ejemplo anterior), y hay al menos un **caso recursivo** (instrutivo) (caso 2 en el ejemplo anterior) que define la respuesta en términos de la respuesta a la pregunta sobre alguna otra entrada, típicamente una versión más simple del mismo problema. Es la presencia de un caso base lo que evita que una definición recursiva sea una definición circular.

La definición recursiva más simple del mundo es probablemente la función factorial (típicamente escrita en matemáticas usando $!$) sobre números naturales. La definición **inductiva** clásica es:

$$1! = 1$$

$$(n + 1)! = (n + 1) * n!$$

La primera ecuación define el caso base. La segunda ecuación define el factorial para todos los números naturales, excepto el caso base, en términos del factorial del número anterior. El siguiente código contiene tanto una implementación iterativa (`fact_iter`) como una recursiva (`fact_rec`) del factorial.

Implementaciones iterativa y recursiva del factorial

```
1  def fact_iter(n):
2      """Assumes n an int > 0
3          Returns n!"""
4      result = 1
5      for i in range(1, n+1):
6          result *= i
7      return result
8
9  def fact_rec(n):
10     """Assumes n an int > 0
11         Returns n!"""
12     if n == 1:
13         return n
14     else:
15         return n * fact_rec(n - 1)
```

Esta función es suficientemente simple que ninguna implementación es difícil de seguir. Aún así, la segunda es una traducción más directa de la definición recursiva original.

Casi parece como hacer trampa implementar `fact_rec` llamando a `fact_rec` desde dentro del cuerpo de `fact_rec`. Funciona por la misma razón que funciona la implementación iterativa. Sabemos que la iteración en `fact_iter` terminará porque a cada iteración positiva se le resta 1. Esto significa que no puede ser mayor que 1 para siempre. Similarmente, si `fact_rec` es llamada con 1, devuelve un valor sin hacer una llamada recursiva. Cuando hace una llamada recursiva, siempre lo hace con un valor uno menor que el valor con el que fue llamada. Eventualmente, la recursión termina con la llamada `fact_rec(1)`.

Ejercicio manual

La suma armónica de un entero, $n > 0$, puede ser calculada usando la fórmula $1 + 1/2 + \dots + 1/n$. Escribe una función recursiva que compute esto.

```
1  def sum_arm(n):
2      if n == 1:
```

```
3         return 1
4     else:
5         return 1 / n + sum_arm(n - 1)
```

Números de Fibonacci

La **secuencia de Fibonacci** es otra función matemática común que usualmente se define recursivamente. A menudo se usan "conejos que se reproducen" para describir una población que el hablante piensa que está creciendo demasiado rápidamente. En el año 1202, el matemático italiano **Leonardo de Pisa**, también conocido como **Fibonacci**, desarrolló una fórmula para cuantificar esta noción, aunque con algunas suposiciones no terriblemente realistas.

Supón que una pareja recién nacida de conejos, un macho y una hembra, son puestos en un corral (o peor, liberados en la naturaleza). Supón además que los conejos pueden aparearse a la edad de un mes (lo cual, sorprendentemente, algunas razas pueden) y tienen un período de gestación de un mes (lo cual, sorprendentemente, algunas razas tienen). Finalmente, supón que estos míticos conejos nunca mueren, y que la hembra siempre produce una nueva pareja (un macho, una hembra) cada mes a partir de su segundo mes. ¿Cuántas hembras conejas habrá al final de seis meses?

En el último día del primer mes (llamémoslo mes 0), habrá una hembra (aún demasiado joven para concebir en el primer día del próximo mes). En el último día del segundo mes, aún habrá solo una hembra (ya que ella no dará a luz hasta el primer día del próximo mes). En el último día del próximo mes, habrá dos hembras (una embarazada y una no). En el último día del próximo mes, habrá tres hembras (dos embarazadas y una no). Y así sucesivamente. Veamos esta progresión en forma tabular.

Crecimiento en la población de conejas hembras

Mes	Mujeres
0	1
1	1
2	2
3	3
4	5

Mes	Mujeres
5	8
6	13

Observa que para el mes $n > 1$, $females(n) = females(n-1) + females(n-2)$. Esto no es un accidente. Cada hembra que estaba viva en el mes $n-1$ seguirá estando viva en el mes n . Además, cada hembra que estaba viva en el mes $n-2$ producirá una nueva hembra en el mes n . Las nuevas hembras se pueden agregar a las hembras del mes $n-1$ para obtener el número de hembras en el mes n .

El crecimiento en la población se describe naturalmente por la **recurrencia**:

```
1  females(0) = 1
2  females(1) = 1
3  females(n + 2) = females(n+1) + females(n)
```

Esta definición es diferente de la definición recursiva del factorial:

- Tiene dos casos base, no solo uno. En general, podemos tener tantos casos base como queramos.
- En el caso recursivo, hay dos llamadas recursivas, no solo una. Nuevamente, puede haber tantas como queramos.

El siguiente código contiene una implementación directa de la recurrencia de Fibonacci, junto con una función que se puede usar para probarla.

Implementación recursiva de la secuencia de Fibonacci

```
1  def fib(n):
2      """Assumes n int >= 0
3          Returns Fibonacci of n"""
4      if n == 0 or n == 1:
5          return 1
6      else:
7          return fib(n-1) + fib(n-2)
8
9  def test_fib(n):
10     for i in range(n+1):
11         print('fib of', i, '=', fib(i))
```

Escribir el código es la parte fácil de resolver este problema. Una vez que pasamos de la declaración vaga de un problema sobre conejitos a un conjunto de ecuaciones recursivas, el código casi se escribió

solo. *Encontrar algún tipo de forma abstracta para expresar una solución al problema en cuestión es a menudo el paso más difícil para construir un programa útil.*

Como podrías suponer, este no es un modelo perfecto para el crecimiento de poblaciones de conejos en la naturaleza. En 1859, **Thomas Austin**, un granjero australiano, importó 24 conejos de Inglaterra para ser usados como objetivos para cazar. Algunos escaparon de Inglaterra para ser usados, aproximadamente dos millones de conejos fueron disparados o atrapados cada año en Australia, sin impacto notable en la población. Esos son muchos conejos, pero no están ni cerca del 120° número de Fibonacci.

Aunque la secuencia de Fibonacci no proporciona realmente un modelo perfecto del crecimiento de poblaciones de conejos, sí tiene muchas propiedades matemáticas interesantes. Los números de Fibonacci también son comunes en la naturaleza. Por ejemplo, para la mayoría de las flores el número de pétalos es un número de Fibonacci.

Ejercicio manual

Cuando la implementación de `fib` es usada para calcular `fib(5)`, ¿Cuántas veces calcula el valor de `fib(2)` en el camino para calcular `fib(5)`?

Tres veces, ya que `fib(5)` llama a `fib(4)` y `fib(3)`. Luego, `fib(4)` a su vez llama a `fib(3)` y `fib(2)`. Ambas llamadas a `fib(3)` (una desde `fib(5)` y otra desde `fib(4)`) invocan a `fib(2)` y `fib(1)`.

Ejercicio manual de la lectura

Implemente la función que cumpla las siguientes especificaciones:

```
1  def recur_power(base, exp):
2      """
3      base: int or float.
4      exp: int >= 0
5
6      Returns base to the power of exp using recursion.
7      Hint: Base case is when exp = 0. Otherwise, in the recursive
8      case you return base * base^(exp-1).
9      """
10     if exp == 0:
```

```
11         return 1
12     else:
13         return base * recur_power(base, exp - 1)
14
15 # Examples:
16 print(recur_power(2,5)) # prints 32
```